

```

"""
algorithms.py – Sorting, Searching, Greedy & Dynamic Programming
=====
Implements:
    1. QuickSort          (Divide & Conquer)
    2. MergeSort          (Divide & Conquer)
    3. Binary Search      (Search)
    4. 0/1 Knapsack       (Dynamic Programming)
    5. Interval Scheduling Maximization (Greedy)
"""

import time
import random
import sys

# -----
# 1. QUICKSORT
# -----

def quicksort(arr: list) -> list:
    """
    In-place Lomuto partition quicksort.
    Average:  $O(n \log n)$  | Worst:  $O(n^2)$  | Space:  $O(\log n)$ 
    """
    def _qs(a, lo, hi):
        if lo < hi:
            p = _partition(a, lo, hi)
            _qs(a, lo, p - 1)
            _qs(a, p + 1, hi)

    def _partition(a, lo, hi):
        # Median-of-three pivot to mitigate worst-case
        mid = (lo + hi) // 2
        candidates = [(a[lo], lo), (a[mid], mid), (a[hi], hi)]
        candidates.sort(key=lambda x: x[0])
        pivot_idx = candidates[1][1]
        a[pivot_idx], a[hi] = a[hi], a[pivot_idx]
        pivot = a[hi]
        i = lo - 1
        for j in range(lo, hi):
            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]
        a[i + 1], a[hi] = a[hi], a[i + 1]
        return i + 1

    result = arr[:]
    _qs(result, 0, len(result) - 1)
    return result

```

```
# -----
# 2. MERGESORT
# -----

def mergesort(arr: list) -> list:
    """
    Top-down recursive merge sort.
    All cases:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    if len(arr) <= 1:
        return arr[:]

    mid = len(arr) // 2
    left = mergesort(arr[:mid])
    right = mergesort(arr[mid:])

    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

# -----

# 3. BINARY SEARCH
# -----

def binary_search(arr: list, target) -> int:
    """
    Iterative binary search on a sorted list.
    Returns index of target, or -1 if not found.
    Time:  $O(\log n)$  | Space:  $O(1)$ 
    """
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

# -----

# 4. 0/1 KNAPSACK (Dynamic Programming)
# -----
```

```

def knapsack_01(weights: list, values: list, capacity: int):
    """
    Classic 0/1 Knapsack via bottom-up DP table.
    Returns (max_value, selected_items_indices)
    Time:  $O(n * W)$  | Space:  $O(n * W)$ 
    """
    n = len(weights)
    # dp[i][w] = max value using items 0..i-1 with capacity w
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        w_i, v_i = weights[i - 1], values[i - 1]
        for w in range(capacity + 1):
            dp[i][w] = dp[i - 1][w] # skip item i
            if w_i <= w:
                take = dp[i - 1][w - w_i] + v_i
                if take > dp[i][w]:
                    dp[i][w] = take # take item i

    # Backtrack to find which items were selected
    selected, w = [], capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i - 1)
            w -= weights[i - 1]
    selected.reverse()

    return dp[n][capacity], selected

```

```

# -----
# 5. INTERVAL SCHEDULING MAXIMIZATION (Greedy)
# -----

```

```

def interval_scheduling(intervals: list) -> list:
    """
    Greedy: select maximum number of non-overlapping intervals.
    Each interval is (start, end, label).
    Sort by earliest finish time → always pick locally optimal.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    sorted_ivs = sorted(intervals, key=lambda x: x[1]) # sort by end time
    selected = []
    last_end = float('-inf')

    for iv in sorted_ivs:
        start, end, *_ = iv
        if start >= last_end:
            selected.append(iv)
            last_end = end

    return selected

```

```
# -----  
# RUNTIME MEASUREMENT UTILITY  
# -----  
  
def measure(func, *args, reps=3):  
    """Run func(*args) `reps` times, return (result, avg_seconds)."""  
    times = []  
    result = None  
    for _ in range(reps):  
        t0 = time.perf_counter()  
        result = func(*args)  
        times.append(time.perf_counter() - t0)  
    return result, sum(times) / reps
```